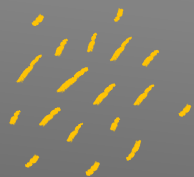


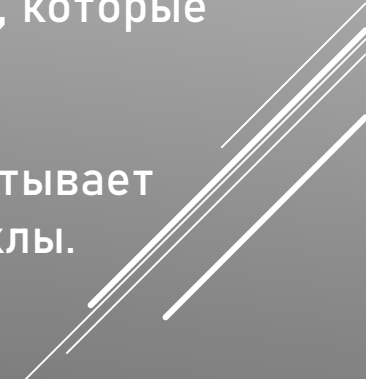
ПРОГРАММИРОВАНИЕ В ЛИНГВИСТИКЕ



Pandas: основы

PANDAS



- **Pandas** – сокращение от panel data.
 - Pandas используется в основном для анализа данных и машинного обучения, но в ней также удобно, например, обрабатывать **выдачу из корпуса** типа ГИКРЯ и любые другие текстовые данные, которые содержатся в таблицах.
 - Pandas – сравнительно быстрая библиотека, потому что основывается на **NumPy**, который гораздо быстрее обрабатывает данные, чем стандартные методы Python, в частности, циклы.
 - С NumPy связаны и **типы данных** Pandas.
- 

Как и в случае numpy, если вы пишете код в pandas, в котором есть **циклы**, очень возможно, это **плохой** код.

DATAFRAME И SERIES

Pandas.DataFrame – основной тип данных в Pandas.

Это таблица, которую можно создать **ручками**, в которую можно преобразовать объекты Python или которую можно получить **при чтении .json, .csv и excel-файлов**.

Pandas.Series – это **колонка или строка** в таблице DataFrame, которую можно получить путем обращения к столбцу с помощью квадратных скобок, а также с помощью методов **.loc** и **.iloc**.

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund; Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings; Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen; Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle; Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen; Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

DataFrame 'Титаник' в .ipynb

СТРУКТУРА DATAFRAME

В отличие от обычных таблиц, в DataFrame строки и столбцы – это **неравноценные** сущности. Проще всего воспринимать **строки как объекты**, а **столбцы как параметры** этих объектов.

Исходя из этой неравноценности, данные в Pandas **типизированы по столбцам**.

Объекты

Параметры

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund; Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings; Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen; Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle; Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen; Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

СОЗДАНИЕ DATAFRAME

DataFrame можно создать многими способами.

1. Преобразованием из **списка списков**:

```
df1 = pd.DataFrame([[1, 2, 3], [4, 5, 6]])  
df1
```

	0	1	2
0	1	2	3
1	4	5	6

2. Преобразованием из **списка словарей**:

```
df2 = pd.DataFrame([{'col1': 1, 'col2': 2},  
                    {'col1': 3, 'col2': 4}])  
df2
```

	col1	col2
0	1	2
1	3	4

СОЗДАНИЕ DATAFRAME

3. Преобразованием из **словаря списков**:

```
df3 = pd.DataFrame({'col1': [1, 2, 3],  
                    'col2': [4, 5, 6]})  
df3
```

	col1	col2
0	1	4
1	2	5
2	3	6

В ключах имена столбцов.

4. Преобразованием из **словаря словарей**:

```
df4 = pd.DataFrame({'col1': {'l1': 1, 'l2': 2},  
                    'col2': {'l1': 3, 'l2': 4}})  
df4
```

	col1	col2
l1	1	3
l2	2	4

Во внешних ключах имена столбцов, во внутренних – имена строк.

СОЗДАНИЕ DATAFRAME

5. Открытием **.csv**, **.json** и **excel**-файлов (последнее требует установки **openpyxl**).

```
[ ] data = pd.read_json(path)
    data = pd.read_csv(path)
    data = pd.read_excel(path)
```

	A	B	C	D	E
1		Сем1	ДЗ1	Сем2	ДЗ2
2	Вася	2	3	1	5
3	Катя	10	10	9	10
4	Петя	5	7	10	6
5	Витя	4	2	6	1



```
df = pd.read_excel('marks.xlsx', index_col=0)
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	2	3	1	5
Катя	10	10	9	10
Петя	5	7	10	6
Витя	4	2	6	1

ПОЛУЧЕНИЕ SERIES

Мы можем получить **столбец** из нашего DataFrame.

DataFrame по способу обращения напоминает **списки и словари**:

```
df['Сем1']
```

Вася	2
Катя	10
Петя	5
Витя	4

Name: Сем1, dtype: int64

Можно обращаться также **через точку** от имени таблицы:

```
df.Сем1
```

Вася	2
Катя	10
Петя	5
Витя	4

Name: Сем1, dtype: int64

Pandas иногда ругается на **кириллицу** в названиях столбцов, но современный работает с ней

ПОЛУЧЕНИЕ SERIES

Также мы можем получить строку из DataFrame с помощью `.loc` и `.iloc`.

Мы можем обратиться по имени строки (здесь это str, а могут быть числа):

```
df.loc['Катя']
```

Сем1	10
Д31	10
Сем2	9
Д32	10

Name: Катя, dtype: int64

И по индексу строки:

```
df.iloc[1]
```

Сем1	10
Д31	10
Сем2	9
Д32	10

Name: Катя, dtype: int64

У любой таблицы можно посмотреть количество ее строк с помощью `len()`

ПОЛУЧЕНИЕ ЗНАЧЕНИЯ ЯЧЕЙКИ

Мы можем запросить содержимое определенной ячейки.

С помощью **имен столбцов и строк**:

```
df['Д32']['Петя']
```

6

С помощью **индексов** (строка, столбец):

```
df.iloc[3, 2]
```

6

Общее количество элементов таблицы (включая NaN и пустые ячейки, если они есть) можно посмотреть с помощью **.size**

ФУНКЦИОНАЛ ДЛЯ ПРОСМОТРА DATAFRAME

<code>DataFrame.head(int)</code>	Показывает такое-то количество строк DataFrame с начала . По умолчанию - 5.
<code>DataFrame.tail(int)</code>	Показывает такое-то количество строк DataFrame с конца . По умолчанию - 5.
<code>DataFrame.sample(int)</code>	Показывает такое-то количество строк DataFrame случайным образом . По умолчанию показывает одну случайную строку.
<code>DataFrame.index</code>	Показывает названия всех строк (индексы) в таблице.
<code>DataFrame.columns</code>	Показывает названия всех столбцов в таблице.
<code>DataFrame.info()</code>	Выводит список всех столбцов , считает, сколько в них непустых значений , и указывает, какого типа данные в них лежат.
<code>DataFrame.describe()</code>	Выводит для каждого из цифровых столбцов таблицы число непустых значений , их среднее арифметическое , стандартное отклонение , минимальное и максимальное значения , а также квантили .

* Квантиль – это значение, которое заданная случайная величина не превышает с фиксированной вероятностью.

ДОБАВЛЕНИЕ СТРОКИ В DATAFRAME

Мы можем добавить **строку** в наш DataFrame.

```
df.loc['Маша'] = [1, 1, 0, 3]  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	2	3	1	5
Катя	10	10	9	10
Петя	5	7	10	6
Витя	4	2	6	1
Маша	1	1	0	3



ДОБАВЛЕНИЕ СТОЛБЦА В DATAFRAME

А также **столбец**:

```
df['Сем3'] = [4, 9, 3, 5]  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2	Сем3
Вася	2	3	1	5	4
Катя	10	10	9	10	9
Петя	5	7	10	6	3
Витя	4	2	6	1	5

УДАЛЕНИЕ СТОЛБЦА ИЛИ СТРОКИ

Для многих операций со столбцами или строками в pandas применяется параметр **axis**. Работает так же, как в numpy: если axis=0 (или не указана), то операция производится со **строкой**, а если 1, то со **столбцом**.

```
df.drop('Витя', inplace=True)  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2	Сем3
Вася	2	3	1	5	4
Катя	10	10	9	10	9
Петя	5	7	10	6	3

```
df.drop('Сем3', axis=1, inplace=True)  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	2	3	1	5
Катя	10	10	9	10
Петя	5	7	10	6

Помните, что в jupyter-тетрадках, чтобы отменить изменения в DataFrame, нужно **перезапустить ядро** («Среда выполнения» -> «Перезапустить сеанс»).

ПАРАМЕТР INPLACE

У многих методов pandas присутствует параметр **inplace**. Как мы помним, in-place методы меняют объект **'на месте'**, а не in-place методы возвращают копию, которую нужно чему-то присвоить.

```
df = df.drop('Сем3', axis=1)  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	2	3	1	5
Катя	10	10	9	10
Петя	5	7	10	6

Без inplace=True

```
df.drop('Сем3', axis=1, inplace=True)  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	2	3	1	5
Катя	10	10	9	10
Петя	5	7	10	6

С inplace=True

ПРИМЕНЕНИЕ УСЛОВИЯ К DATAFRAME

Можно попросить Pandas показать нам только ту часть DataFrame, которая **удовлетворяет какому-либо условию**.

```
df[df > 5].dropna()
```

	Сем1	ДЗ1	Сем2	ДЗ2
Катя	10.0	10.0	9.0	10.0

Покажет отличника (метод **.dropna()** удаляет пустые значения и NaN)

```
df[(df['ДЗ1'] > 5) & (df['ДЗ2'] > 5)]
```

	Сем1	ДЗ1	Сем2	ДЗ2
Катя	10	10	9	10
Петя	5	7	10	6

Покажет, кто хорошо решал домашки (соединяем условия **как булевы маски**)

NaN (Not a Number) означает, что в ячейке **числового** столбца нет данных.

ПРИМЕНЕНИЕ УСЛОВИЯ К DATAFRAME

Применяя условия, можно работать со значениями отдельно у строк и столбцов.

Такое условие создаст NaN в ячейках, где оно не сработало.

```
df = df[df > 3]  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	NaN	NaN	NaN	5.0
Катя	10.0	10.0	9.0	10.0
Петя	5.0	7.0	10.0	6.0
Витя	4.0	NaN	6.0	NaN

Мы можем посмотреть, есть ли в наших столбцах NaN:

```
df.isna().any(axis=1)
```

Вася	True
Катя	False
Петя	False
Витя	True
dtype:	bool

И удалить все строки, в которых встречается хотя бы один NaN.

```
df.dropna(inplace=True)  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Катя	10.0	10.0	9.0	10.0
Петя	5.0	7.0	10.0	6.0

УНИКАЛЬНЫЕ ЗНАЧЕНИЯ В DATAFRAME

Мы можем посмотреть, **какие уникальные** оценки заработала Катя:

```
df.loc['Катя'].unique()  
  
array([10,  9])
```

И посчитать **количество** таких уникальных оценок.

```
df.loc['Катя'].nunique()  
  
2
```

*В таких маленьких DataFrame все видно визуально, но представьте себе DataFrame размером в 5000 строк!

ОПЕРАЦИИ НАД DATAFRAME

С **числовым** DataFrame можно совершать любые математические операции самым простым образом – так же, **как с массивом numpy**.

```
df += 1  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	3	4	2	6
Катя	11	11	10	11
Петя	6	8	11	7
Витя	5	3	7	2

Добавляем всем студентам балл

```
df['Результат'] = df.sum(axis=1).astype(int)  
df['Зачет'] = (df['Результат'] / 100 * 30).astype(int)  
df
```

	Сем1	ДЗ1	Сем2	ДЗ2	Результат	Зачет
Вася	2	3	1	5	11	3
Катя	10	10	9	10	39	11
Петя	5	7	10	6	28	8
Витя	4	2	6	1	13	3

Добавляем столбец 'Зачет' на основе набранных баллов

ПРИМЕНЕНИЕ ФУНКЦИИ К DATAFRAME

В качестве замены циклам у DataFrame есть метод `.apply`, по принципу напоминающий ванильный `map()`: он позволяет **применить функцию** ко всему DataFrame, его столбцу или строке.

```
df = df.apply(lambda x: x + 1)
df
```

	Сем1	ДЗ1	Сем2	ДЗ2
Вася	3	4	2	6
Катя	11	11	10	11
Петя	6	8	11	7
Витя	5	3	7	2

Применение ко всему
DataFrame

```
df['Результат'] = df.sum(axis=1).astype(int)
df['Зачет'] = df['Результат'].apply(lambda x: x / 100 * 30).astype(int)
df
```

	Сем1	ДЗ1	Сем2	ДЗ2	Результат	Зачет
Вася	2	3	1	5	11	3
Катя	10	10	9	10	39	11
Петя	5	7	10	6	28	8
Витя	4	2	6	1	13	3

Применение к столбцу, результат кладется в
дополнительный столбец

УДАЛЕНИЕ ДУБЛИКАТОВ

Если у нас в DataFrame есть повторяющиеся строки или столбцы, мы можем удалить их с помощью метода `.drop_duplicates()`.

```
df.drop_duplicates(keep='first', inplace=True)
```

Параметр `keep` определяет, какие из повторяющихся строк нужно оставить, а какие удалить. Варианты: `'first'`, `'last'`, `'False'` (последнее удаляет все дубликаты, оставляя только изначально уникальные строки), по умолчанию `'first'`.

Такой метод хорош для чистки больших DataFrame – например, выдач из корпусов типа ГИКРЯ.

АГРЕГАЦИЯ: GROUPBY

Нам может понадобиться сгруппировать данные в нашей таблице, чтобы получить еще больше полезной информации.

Самый простой способ группировки – метод **groupby**. Он позволяет группировать данные **по любым столбцам**, но этого часто может оказаться мало.

```
[44] titanic.groupby('Sex').mean(numeric_only=True)
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
Sex							
female	461.818182	0.931818	1.215909	32.676136	0.534091	0.545455	89.000900
male	449.389474	0.431579	1.168421	38.451789	0.400000	0.410526	69.124343

АГРЕГАЦИЯ: СУММА И СРЕДНЕЕ

Мы можем посчитать **сумму** с помощью метода **.sum()** и вычислить **среднее арифметическое** для с помощью **.mean()**:

Оба метода могут применяться и к **отдельным столбцам**, и к **результату .groupby()**.

Также есть **.median()**, **.min()**, **.max()** и **.std()**, а еще **.count()** – количество заполненных значений, **.size()** – размер группы, и др.

```
df['Результат'] = df.sum(axis=1).astype(int)
df['Средн. арифм.'] = df.mean(axis=1).astype(int)
df
```

	Сем1	Д31	Сем2	Д32	Результат	Средн. арифм.
Вася	2	3	1	5	11	4
Катя	10	10	9	10	39	15
Петя	5	7	10	6	28	11
Витя	4	2	6	1	13	5

АГРЕГАЦИЯ: PIVOT_TABLE

```
pd.pivot_table(df,  
               index=["Manager", "Rep"],  
               values=["Price"],  
               aggfunc=[np.mean, len])
```

		mean	len
		Price	Price
Manager	Rep		
Debra Henley	Craig Booker	20000.000000	4
	Daniel Hilton	38333.333333	3
	John Smith	20000.000000	2
Fred Anderson	Cedric Moss	27500.000000	4
	Wendy Yule	44250.000000	4

Более продвинутый способ группировки – метод **pivot_table** (сводная таблица).

Он обладает множеством параметров, что позволяет удобно и качественно сгруппировать данные произвольным образом, а также посчитать для них любые функции агрегации.

Больше узнать про `pivot_table`, а также про другие методы Pandas, поможет простая и доступная [документация](#).

Не забывайте и про [Stack Overflow](#), где можно найти решение ваших проблем.

ПОДСЧЕТ ЗНАЧЕНИЙ

Метод `value_counts()` применяется к столбцу или строке и возвращает Series в формате '[элемент] [встретился столько раз]'.

Метод является удобной альтернативой использованию `.size()` с результатом `.groupby()`.

Связка `.groupby()` плюс `.size()` может быть предпочтительной, например, чтобы **результат остался в виде таблицы** с красивыми заголовками

```
df3 = pd.read_csv('titanic.csv', sep=';')
```

```
df3['Sex'].value_counts()
```

male	574
female	312
unknown	5

Сколько на Титанике было женщин и мужчин



Спасибо за внимание!

